UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingenierías
Ingeniería en Desarrollo de Software y Ciencia de Datos
Bases de Datos II (No Relacionales)

PROYECTO FINAL
VETAGENDA NOSQL

Sistema de agenda veterinaria, control de vacunas y reportes con MongoDB + API REST

Integrantes: _____

_____

_____

Catedrático: Mg. Josué A. Guinea Henríquez

Octubre de 2026

1. Descripción del proyecto

VetAgenda NoSQL es un sistema para una clínica veterinaria que integra el registro de clientes y mascotas, el historial básico de vacunas, la programación de citas y la generación de reportes. La idea principal es funcionar como una agenda clínica: permite consultar qué citas hay en un día y qué mascotas tienen próximas dosis de vacuna, sin limitarse a un CRUD de una sola colección.

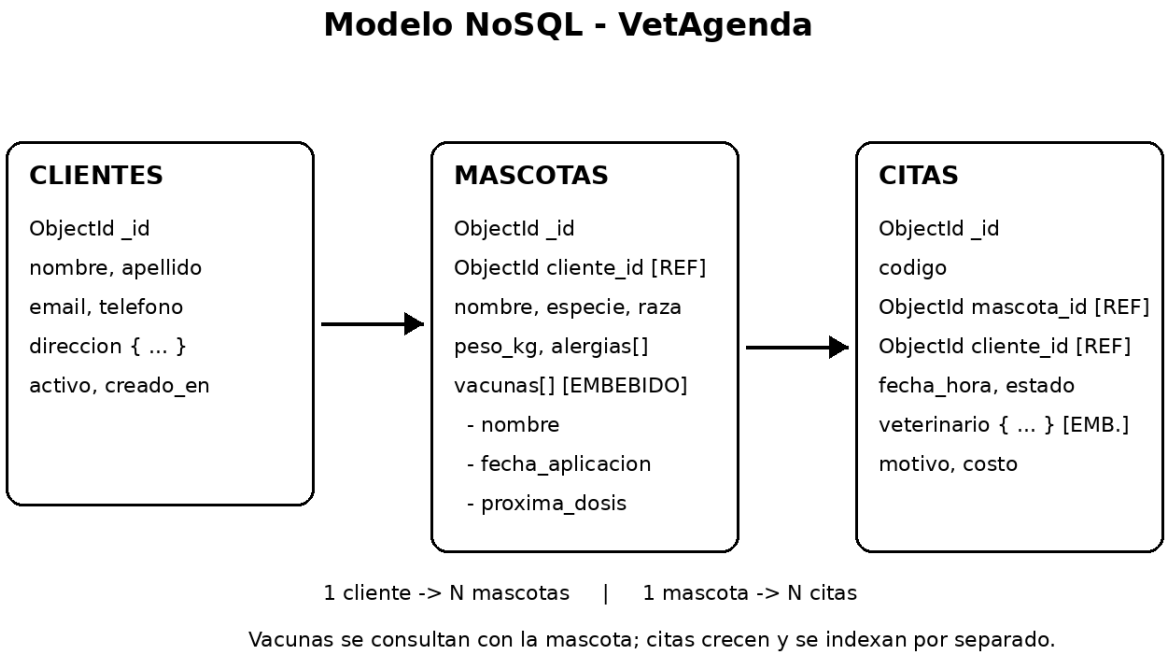
La API incorpora una regla de negocio adicional: al programar una cita verifica que no exista un cruce de horario para la misma mascota o para el mismo veterinario. Esto evita duplicidad de agenda y demuestra lógica de negocio sobre los datos.

2. Arquitectura seleccionada

Tecnologías: MongoDB 7, Node.js 20, Express y el driver oficial mongodb. La aplicación expone una API REST y puede levantarse con Docker Compose. MongoDB fue seleccionado porque el caso necesita documentos con subestructuras naturales (vacunas y datos del veterinario) y, al mismo tiempo, referencias entre entidades que crecen de forma independiente.

Componente	Tecnología	Función
Base de datos	MongoDB	Persistencia NoSQL, validaciones, índices y aggregation pipelines.
Backend	Node.js + Express	Endpoints REST, reglas de negocio y paginación.
Entorno	Docker Compose	Levanta MongoDB, inicializa/siembra datos y arranca la API.
Pruebas	Postman	Colección de peticiones para comprobar la API.

3. Modelo de datos NoSQL



3.1 Colección clientes

Almacena la información de contacto del propietario. Se mantiene separada porque un cliente puede tener varias mascotas y sus datos no deberían duplicarse en cada documento de mascota.

3.2 Colección mascotas

Cada mascota referencia a su propietario mediante cliente_id. El arreglo vacunas se embebe dentro de la mascota porque pertenece directamente a su historial, normalmente se consulta junto con ella y cada elemento tiene una estructura pequeña y estable: nombre, fecha aplicada, próxima dosis, lote y veterinario.

3.3 Colección citas

Las citas se almacenan aparte porque son la información de mayor crecimiento y se consultan principalmente por fecha, estado, mascota, cliente y veterinario. Guardan referencias a mascota y cliente. El veterinario se almacena como un pequeño snapshot embebido para conservar el dato histórico visible de la cita sin crear una cuarta colección innecesaria para el alcance del proyecto.

4. Validación con $jsonSchema

El script scripts/init-db.js crea las tres colecciones con validationLevel strict y validationAction error. De esta forma MongoDB rechaza documentos que no cumplan la estructura esperada.

clientes: Obliga nombre, apellido, correo, teléfono, dirección, estado y fecha de creación. El correo debe tener un formato básico válido.

mascotas: Exige ObjectId de cliente, especie permitida, sexo, fecha de nacimiento, peso positivo y arreglo de vacunas con fechas reales.

citas: Valida código, referencias ObjectId, fecha, duración entre 15 y 180 minutos, estado permitido, veterinario y costo no negativo.

5. Índices

Colección	Índice	Motivo
clientes	email (unique)	Evita correos duplicados y acelera búsqueda por correo.
clientes	direccion.ciudad + activo	Filtra propietarios por ciudad y estado.
mascotas	cliente_id + activo	Lista mascotas de un propietario.
mascotas	especie + peso_kg	Soporta filtros combinados.
mascotas	vacunas.proxima_dosis	Agenda recordatorios de vacunación.
citas	codigo (unique)	Identificador lógico de cita.
citas	fecha_hora + estado	Agenda diaria y rangos de fechas.
citas	mascota_id + fecha_hora	Historial y detección de conflictos.
citas	veterinario.id + fecha_hora	Agenda del veterinario y conflictos.

6. Consultas avanzadas y reglas de negocio

Los endpoints aceptan filtros con operadores de comparación y lógicos. Ejemplos: peso mínimo/máximo ($gte/$lte), vacunas con próxima dosis antes de una fecha ($elemMatch), búsqueda textual aproximada ($regex), estados permitidos/excluidos ($in/$nin) y alternativas ($or). Las respuestas de listados incluyen paginación y ordenamiento.

Regla de agenda: antes de insertar una cita, la API busca citas activas de la misma mascota o del mismo veterinario alrededor del horario pedido. Luego compara los intervalos usando fecha_hora + duracion_min. Si existe solapamiento devuelve HTTP 409 y no guarda la nueva cita.

7. Aggregation Pipeline principal

El endpoint GET /api/reportes/vacunas-proximas?dias=30 procesa el arreglo de vacunas y genera un reporte resumido. Utiliza las siguientes etapas:

- $unwind: separa temporalmente cada vacuna del arreglo.
- $match: conserva vacunas cuya próxima dosis cae dentro del rango solicitado.
- $lookup: une la mascota con su cliente para obtener propietario y teléfono.
- $group: agrupa por especie y nombre de vacuna, calculando total de mascotas y la fecha más próxima.
- $project: da forma clara al reporte.
- $sort: ordena por fecha de próxima dosis.

También existe GET /api/reportes/citas-mensuales, que agrupa las citas por año y mes y devuelve cantidad de citas, ingresos y ticket promedio.

8. Endpoints y funcionalidades

Método / ruta	Tipo	Objetivo
POST /api/clientes	Escritura	Crear propietario.
GET /api/clientes	Lectura	Filtrar, paginar y ordenar clientes.
POST /api/mascotas	Escritura	Crear mascota referenciada al cliente.
GET /api/mascotas	Lectura avanzada	Filtrar por especie, peso, cliente y próxima vacuna.
POST /api/mascotas/:id/vacunas	Actualización	Agregar una vacuna al arreglo embebido.
POST /api/citas	Escritura + lógica	Crear cita validando cruces de horario.
GET /api/citas	Lectura avanzada	Consultar por rango, estado, veterinario, cliente o mascota.
GET /api/citas/agenda/dia	Agenda integrada	Combina citas del día con próximas vacunas.
GET /api/reportes/vacunas-proximas	Analítica	Pipeline complejo con $lookup/$unwind/$group.
GET /api/reportes/citas-mensuales	Analítica	Resumen mensual de citas e ingresos.

9. Datos iniciales (seed)

El archivo scripts/seed.js limpia los datos anteriores y genera datos ficticios reproducibles: 60 clientes, 80 mascotas y 100 citas. Las mascotas incluyen vacunas y próximas dosis distribuidas en distintas fechas; las citas se distribuyen en varios meses, estados y veterinarios. Esto permite probar filtros, agenda y reportes con volumen suficiente.

10. Ejecución del proyecto

Opción recomendada con Docker:

```
docker compose up --build
```

El servicio setup espera a que MongoDB responda, aplica validadores e índices, carga el seed y después se inicia la API en http://localhost:3000.

Opción local:

```
cp .env.example .env

npm install

npm run setup

npm start
```

11. Pruebas

El archivo docs/VetAgenda.postman_collection.json puede importarse directamente en Postman. Incluye peticiones para clientes, mascotas, vacunas, citas, agenda y los dos reportes. Para operaciones que requieran un ObjectId se deben copiar IDs reales obtenidos de los listados y colocarlos en las variables clienteId, mascotaId y citaId de Postman.

12. Matriz de cumplimiento del enunciado

Requisito	Cumplimiento	Archivo / evidencia
MongoDB con 2-3 colecciones y $jsonSchema	Sí	scripts/init-db.js; clientes, mascotas y citas
Embeber vs. referenciar justificado	Sí	Vacunas embebidas; relaciones por ObjectId; sección 3
Filtros avanzados	Sí	routes/mascotas.js, clientes.js y citas.js
Aggregation Pipeline	Sí	routes/reportes.js
API REST con escritura	Sí	POST clientes, mascotas, vacunas y citas
Lectura, filtros, paginación y ordenamiento	Sí	GET clientes, mascotas y citas
Endpoint de reporte/analítica	Sí	/api/reportes/vacunas-proximas y citas-mensuales
README claro	Sí	README.md
Seed 50-100 por colección	Sí	60 clientes, 80 mascotas, 100 citas
Postman/Swagger	Sí	VetAgenda.postman_collection.j

		son y openapi.yaml

13. Conclusión

VetAgenda demuestra un uso de MongoDB orientado al caso de negocio. Las vacunas se modelan como datos embebidos para favorecer la lectura de la mascota, mientras que citas y propietarios se separan para permitir crecimiento, reutilización e índices específicos. La API añade lógica de agenda, filtros y reportes, por lo que el proyecto va más allá de un CRUD simple y permite probar los principales temas de Bases de Datos II.